



The Strategy Pattern

Human Computer Interaction Research
University of Nevada, Reno



Review

What did we learn from the last exercise?

Today

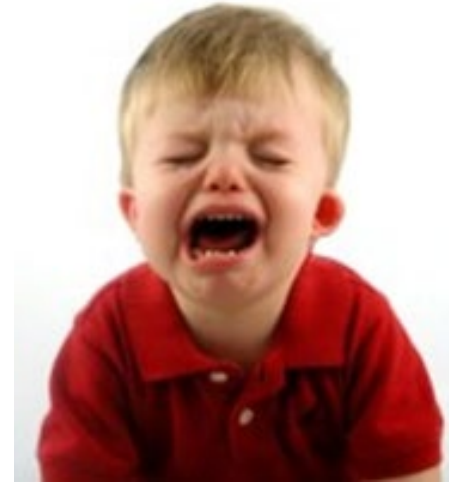
- ☐ Types of Patterns
- ☐ The Strategy Pattern
- ☐ Guitar Hero example
- ☐ RTS exercise

Types of Patterns

- ☐ Behavioral Patterns – interaction or communication between objects
- ☐ Creational patterns - object creation
- ☐ Structural patterns - object and class composition

Behavioral Patterns

- Behavioral design patterns are concerned with **the interaction and responsibility of objects.**
- In these design patterns, **the interaction between the objects should be in such a way that they can easily talk to each other and still should be loosely coupled.**
- These patterns increase **flexibility**



Design Pattern Collection

□ Behavioral Patterns

» "strategy"

problem

Your program needs to
support different kinds of
behavior

Solution

The Strategy Pattern defines a family of algorithms, encapsulates each one and makes them interchangeable. Strategy lets the algorithm vary independently from clients that use it.

example: student records

List

joe

jill

jack

james

joie

jamie

behavior

Quick Sort

Merge Sort

Bubble Sort

contexts

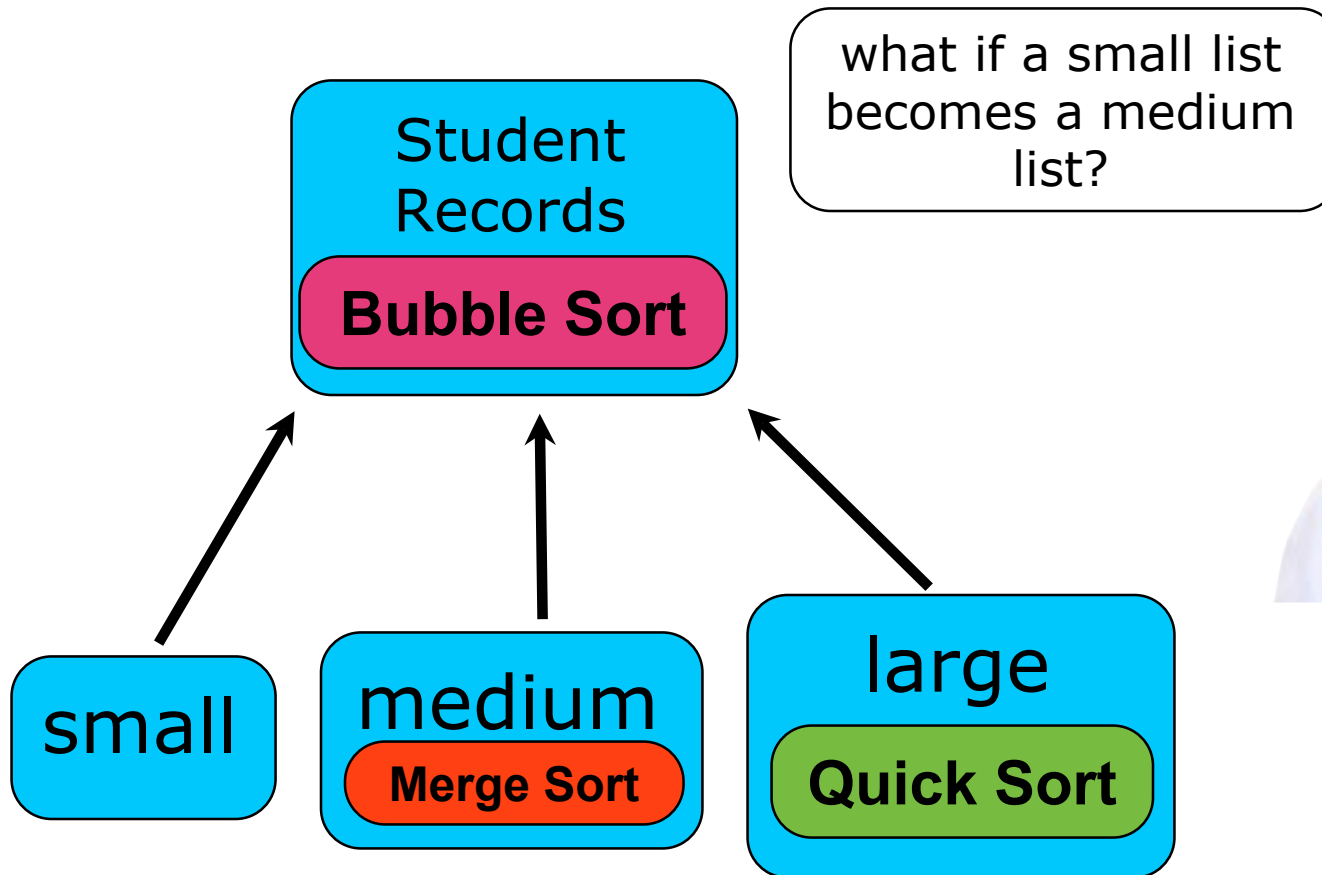
$N > 100$

$50 < N < 100$

$N < 50$

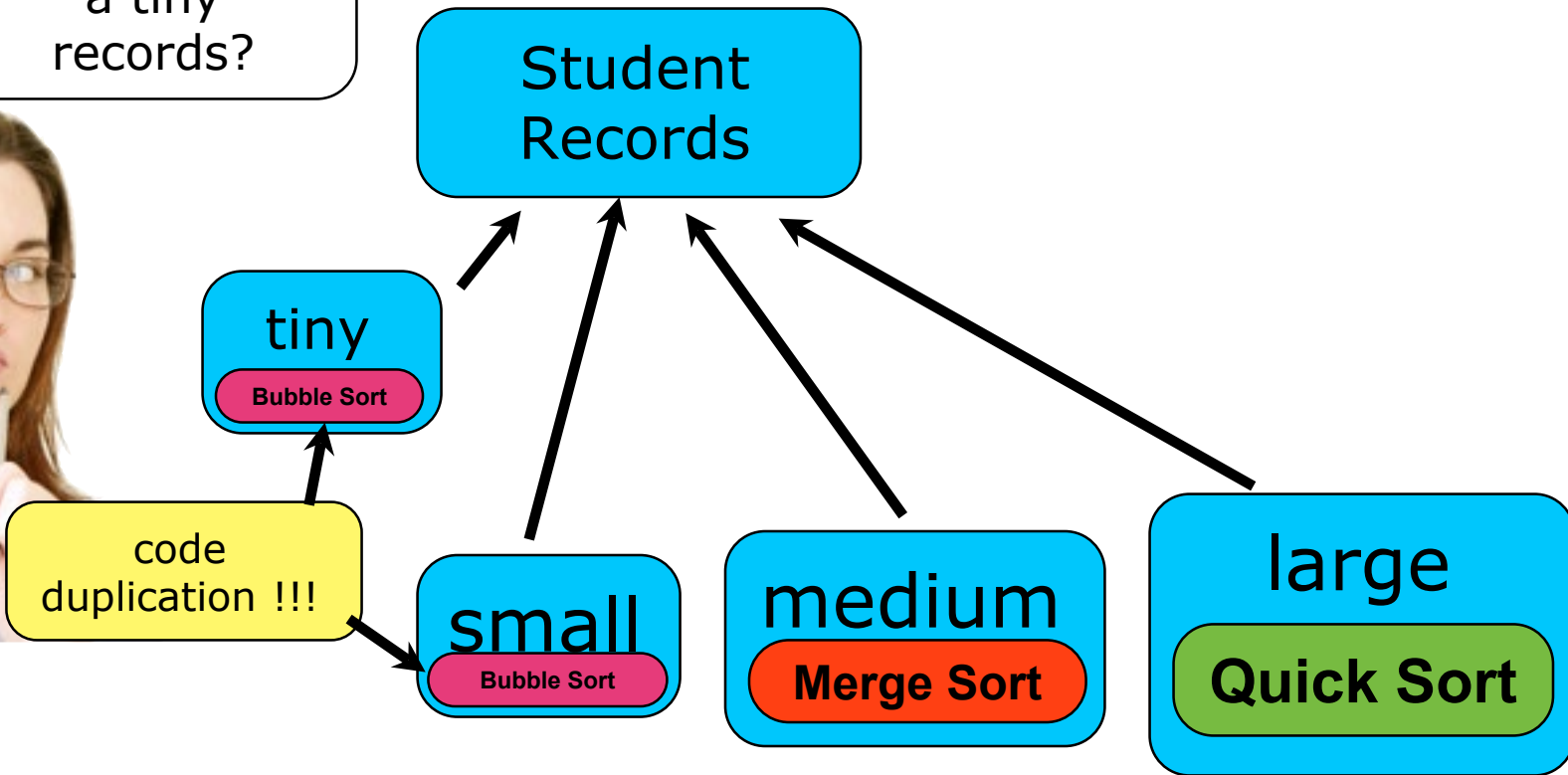
set sorting algorithm
during runtime
depending on context

naive approach



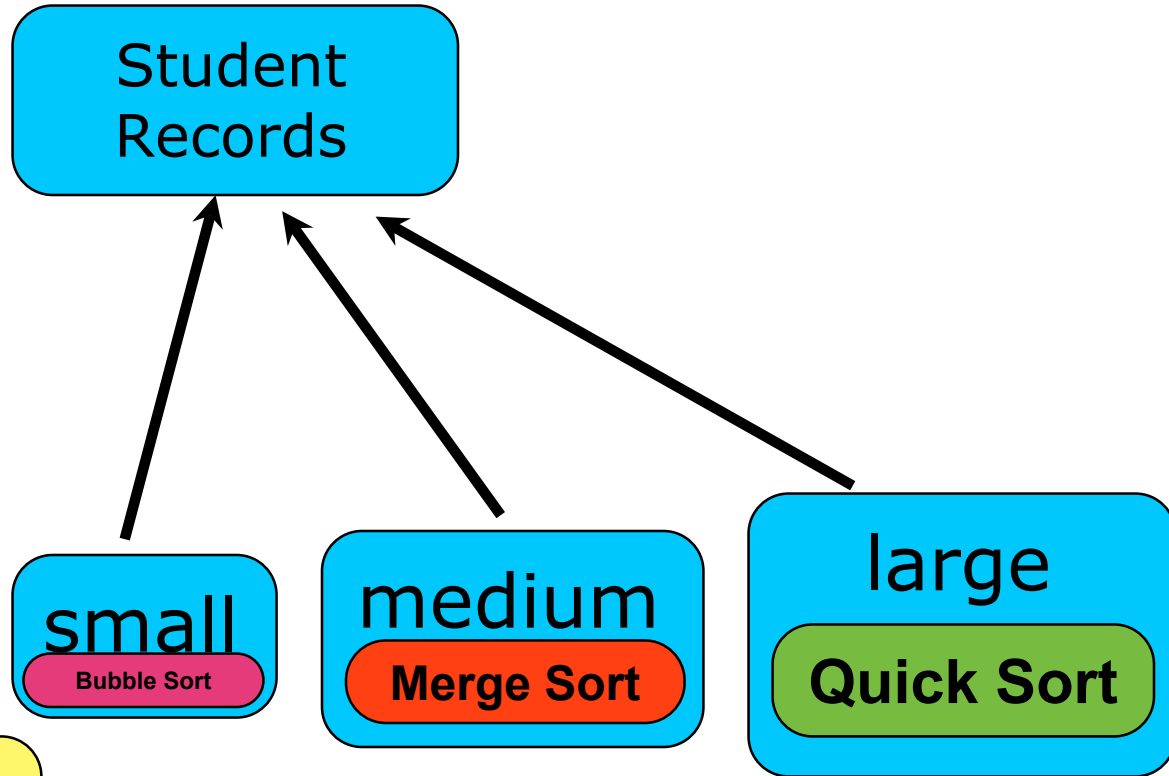
future issues

what if I add
a tiny
records?



future issues

what if I change the student's records list to a hashtable?



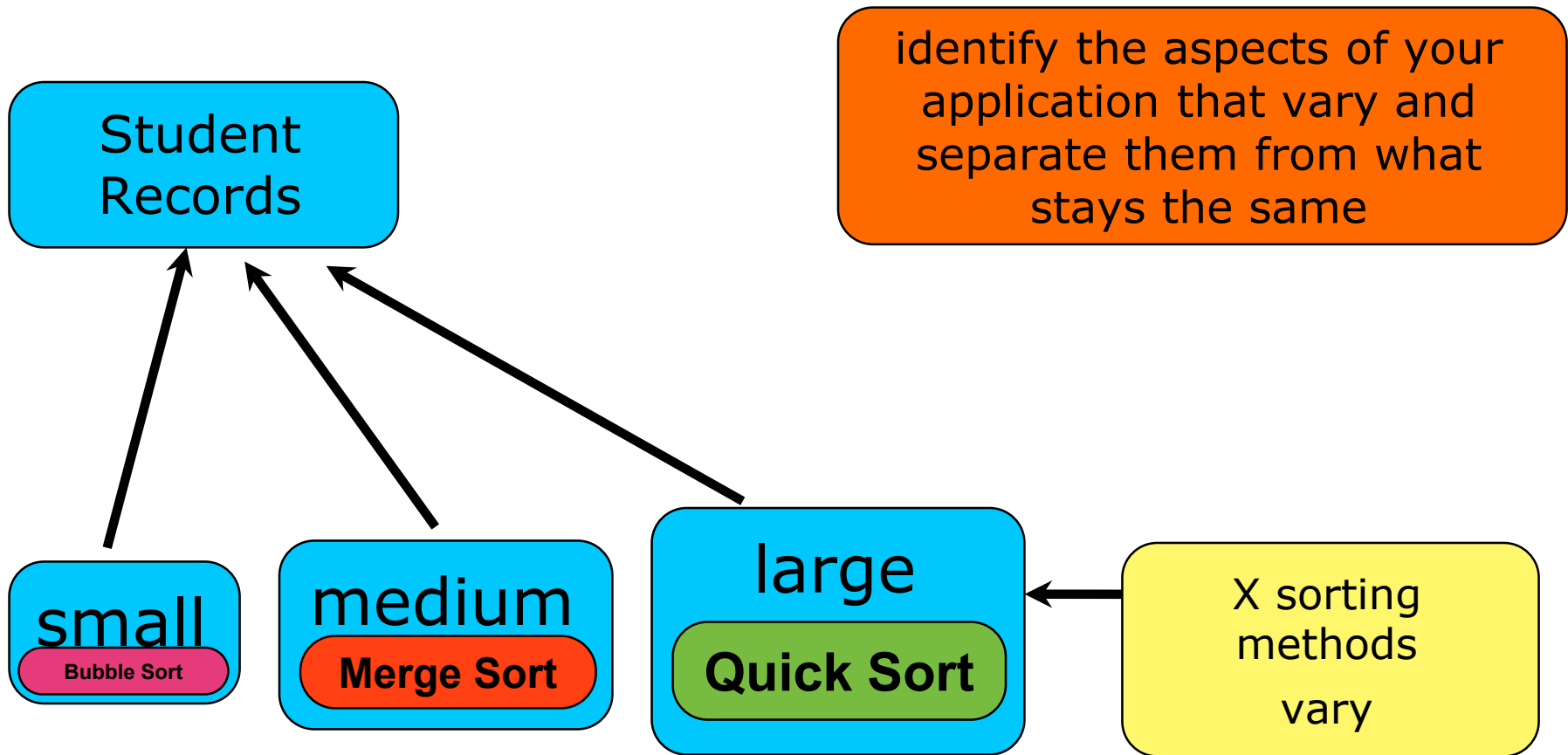
maintenance
nightmare!!

Design Principles

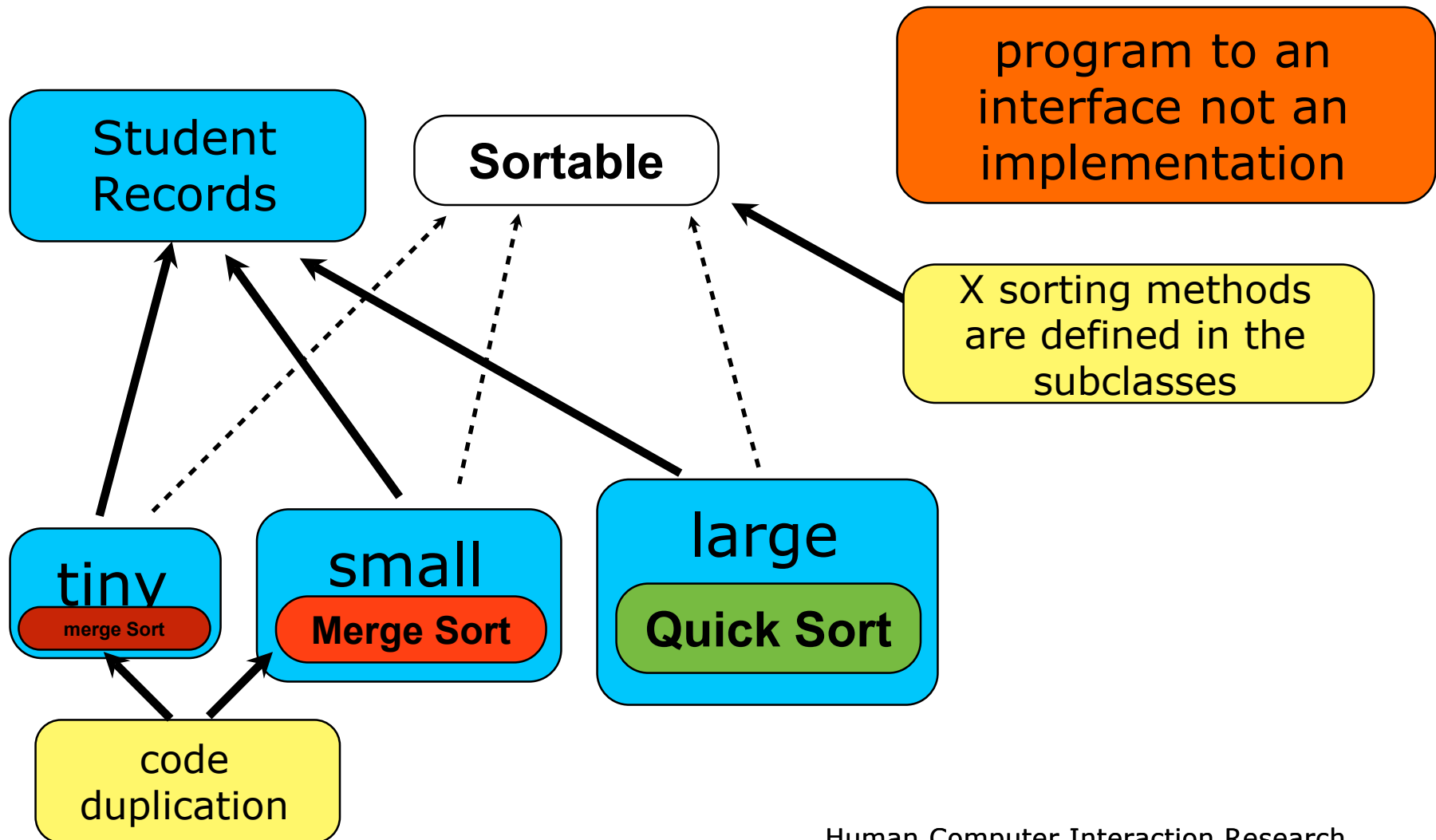
identify the aspects of your application that vary and separate them from what stays the same

program to an interface not an implementation

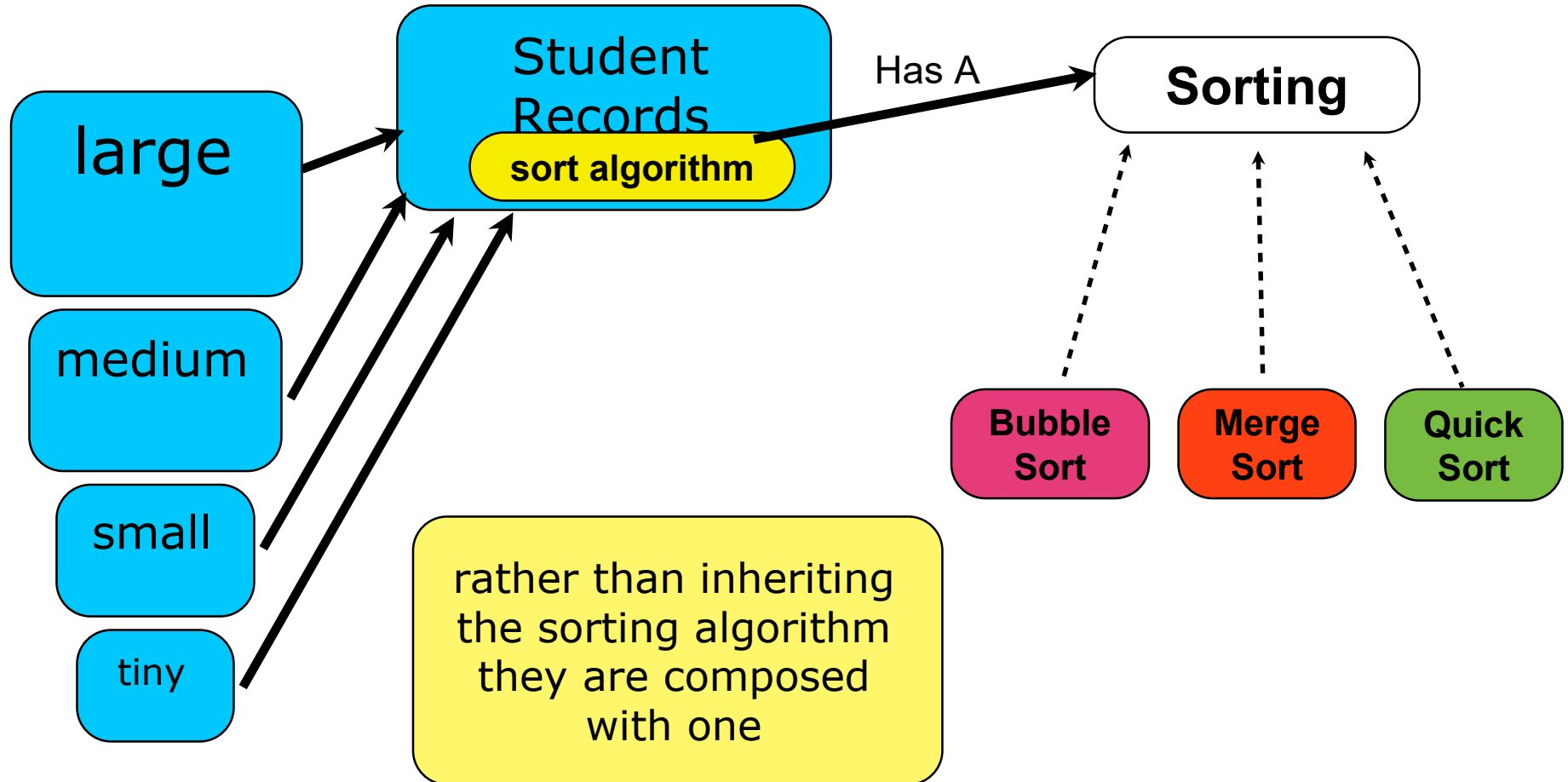
program to interface



program to interface



Program to supertype



Design Principles

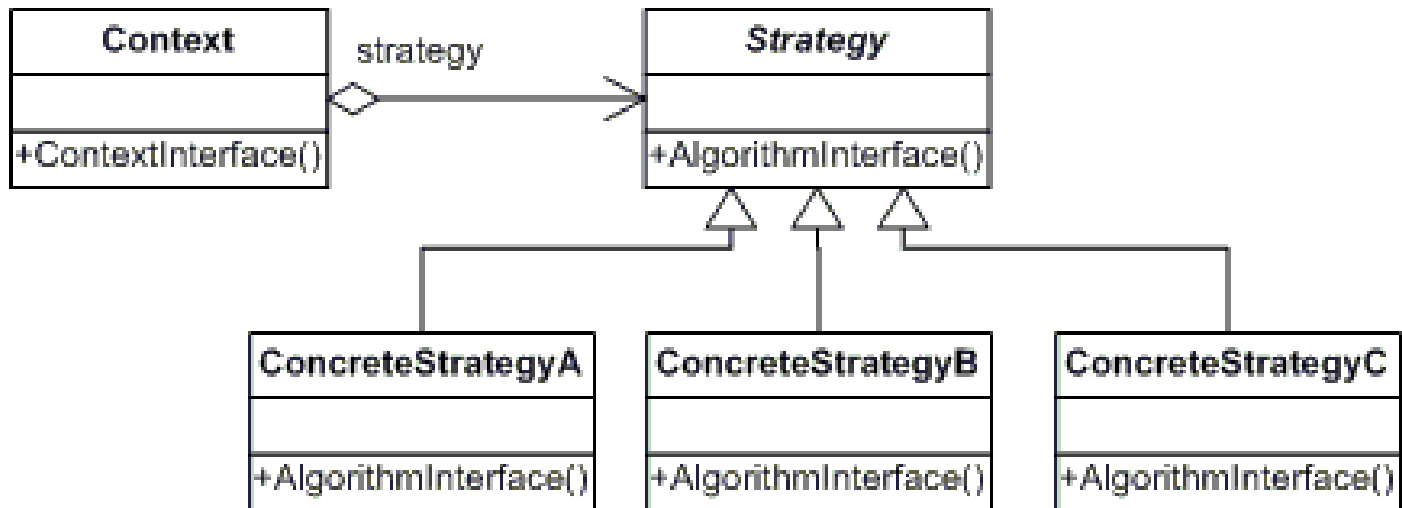
favor composition over inheritance

- ☐ add more flexibility
- ☐ encapsulate family into classes
- ☐ Change behavior at runtime

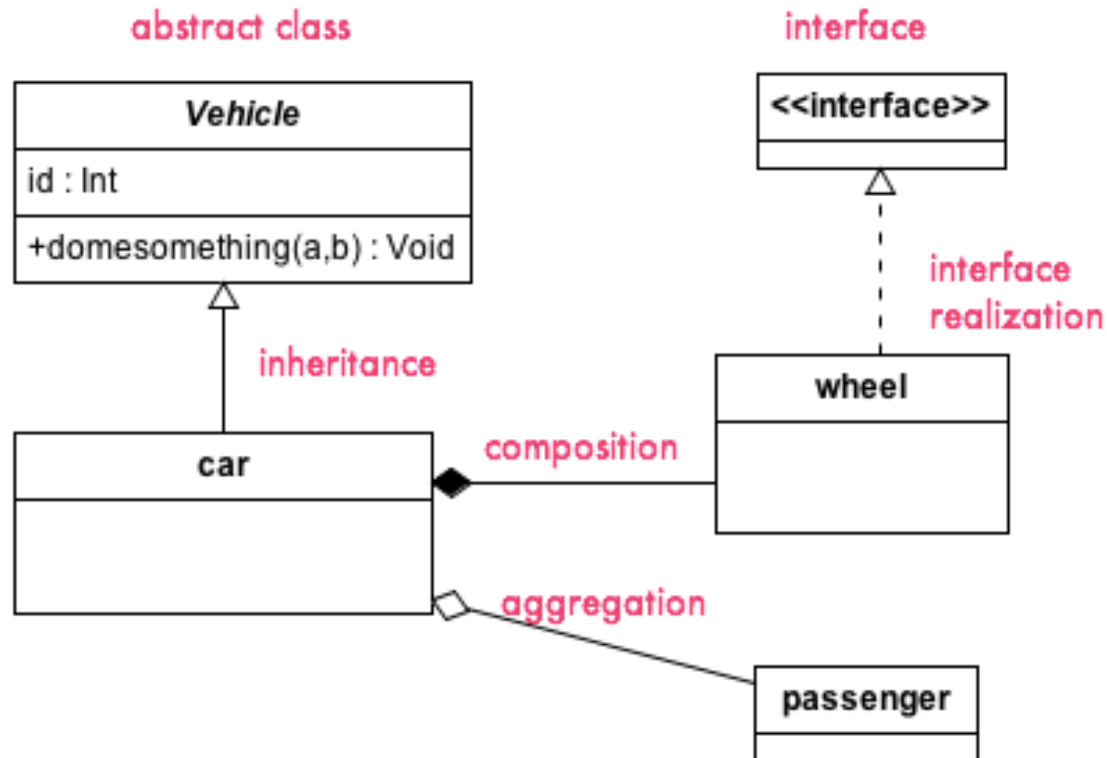
Solution

- The Strategy Pattern first identifies the behaviors or algorithms that vary and separates them from the system that stays the same.
- These behaviors or algorithms are encapsulated in classes that implement a common interface. This enables the developer to program to an interface and not an implementation.

Generic Class Diagram



UML semantics



Participants

- ❑ **Strategy** declares an interface common to all supported algorithms. Context uses this interface to call the algorithm defined by a ConcreteStrategy
- ❑ **ConcreteStrategy** implements the algorithm using the Strategy interface
- ❑ **Context** is configured with a ConcreteStrategy object maintains a reference to a Strategy object may define an interface that lets Strategy access its data.

example: Guitar Hero XIII



The class with the main method

```
public class GuitarHero {  
    public static void main(String[] args) {  
        GameCharacter player1 = new GameCharacterSlash();  
        GameCharacter player2 = new GameCharacterHendrix();  
        player1.playGuitar();  
        player2.playGuitar();  
        player1.playSolo();  
        player2.playSolo();  
    }  
}
```

GameCharacter Class

```
public abstract class GameCharacter {  
    GuitarBehavior guitarBehavior;  
    SoloBehavior soloBehavior;  
  
    public GameCharacter() {  
    }  
  
    public void playGuitar() {  
        guitarBehavior.play();  
    }  
    public void playSolo() {  
        soloBehavior.solo();  
    }  
}
```


Guitar Behavior

```
public interface GuitarBehavior {  
    public void play();  
}
```

```
public class Guitar_GibsonSG implements GuitarBehavior {  
    public void play() {  
        System.out.println("Gibson SG \n");  
    }  
}
```

Solo Behavior

```
public interface SoloBehavior {  
    public void solo();  
}
```

```
public class Solo_JumpOffStage implements SoloBehavior {  
    public void solo() {  
        System.out.println("Jump of Stage\n");  
    }  
}
```

Some Game Characters

```
public class GameCharacterSlash extends GameCharacter {
```

```
    public GameCharacterSlash() {  
        guitarBehavior=new Guitar_GibsonSG();  
        soloBehavior=new Solo_JumpOffStage();  
    }  
}
```

```
public class GameCharacterHendrix extends GameCharacter {
```

```
    public GameCharacterHendrix() {  
        guitarBehavior=new Guitar_Telecaster();  
        soloBehavior=new Solo_SetGuitarOnFire();  
    }  
}
```

Change on the fly

```
public abstract class GameCharacter {
    GuitarBehavior guitarBehavior;
    SoloBehavior soloBehavior;

    // additional functions
    public void setGuitar(GuitarBehavior gb) {
        guitarBehavior=gb;
    }

    public void setSolo(SoloBehavior sb) {
        soloBehavior=sb;
    }
}

// in player
public void change() {
    this.setGuitar(new Guitar_GibsonSG());
}
```

Class diagram Guitar Hero

